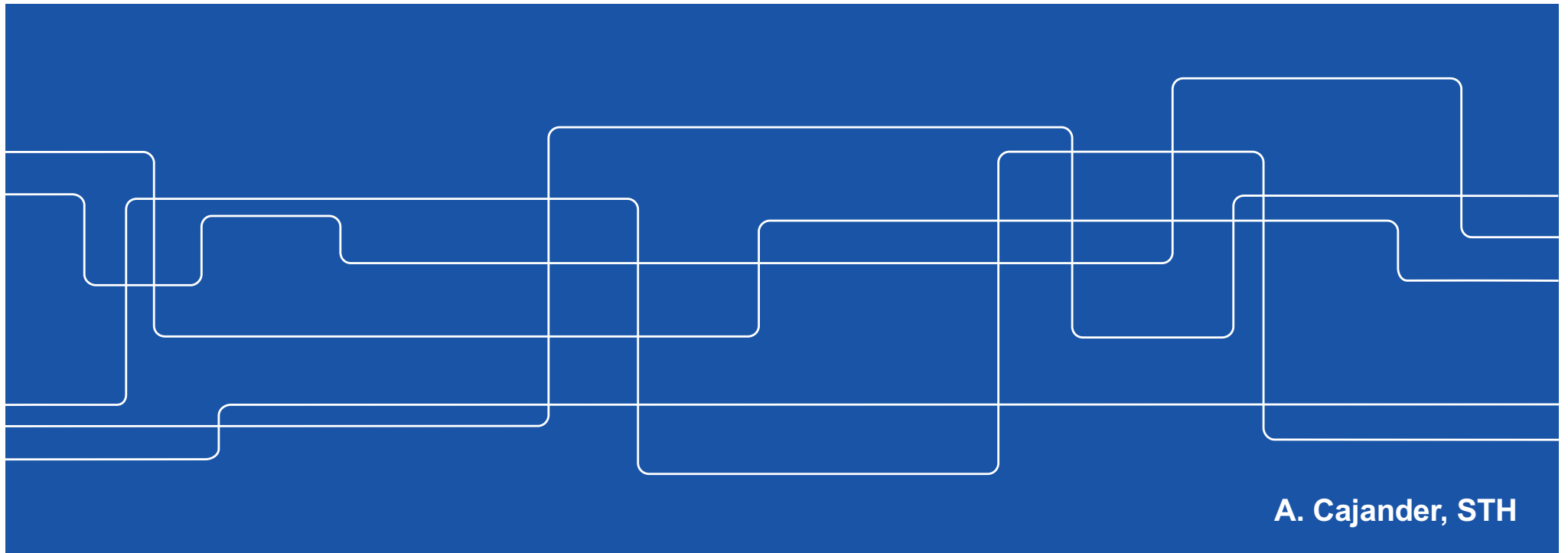




HE1025 Operativsystem

F6: Parallellism – Condition Variabels & Semaphores



A. Cajander, STH



Kursen...

F1: V#1 Processorn.	Kap (1) 2..4 & 6
Ö1: V#2 c Pekare...	Kap 14 & c-boken
F2: V#3 Schemaläggning	Kap 7..9,(10-11), RTOS
L1: V#4 FreeRTOS	Lab #1 Handledning
F3: V#5 Segmentation	Kap 12..13, 15..17
Ö2: V#6 Buffer	t.b.d
F4: V#7 Paging	Kap 18-22, (23-24)
L2: V#8 Paging	Lab #2 Handledning
F5: C#1 Locks & CV	Kap (25) 26..27 & 28..29.1
Ö3: C#2 LINUX	Kap 5 (39)
F6: C#3 Semaphores	Kap 30..33, (34)
L3: C#4 Barberare	Lab #3 Handledning
F7: P#1 Storage	Kap (35,) 36, 40, 42 (37..39, 41 & 43..46)
Ö4: P#2 Buffer	t.b.d
F8: P#3 Communication	Kap (47, 39) & 48, (49..51)
L4: P#4 Client/Server	Lab #4 Handledning

TEN1: 4+4p/4:e-del, minst 4p per 4:e-del för E!

LAB1: Närvaro L1..4



IPC/Concurrency

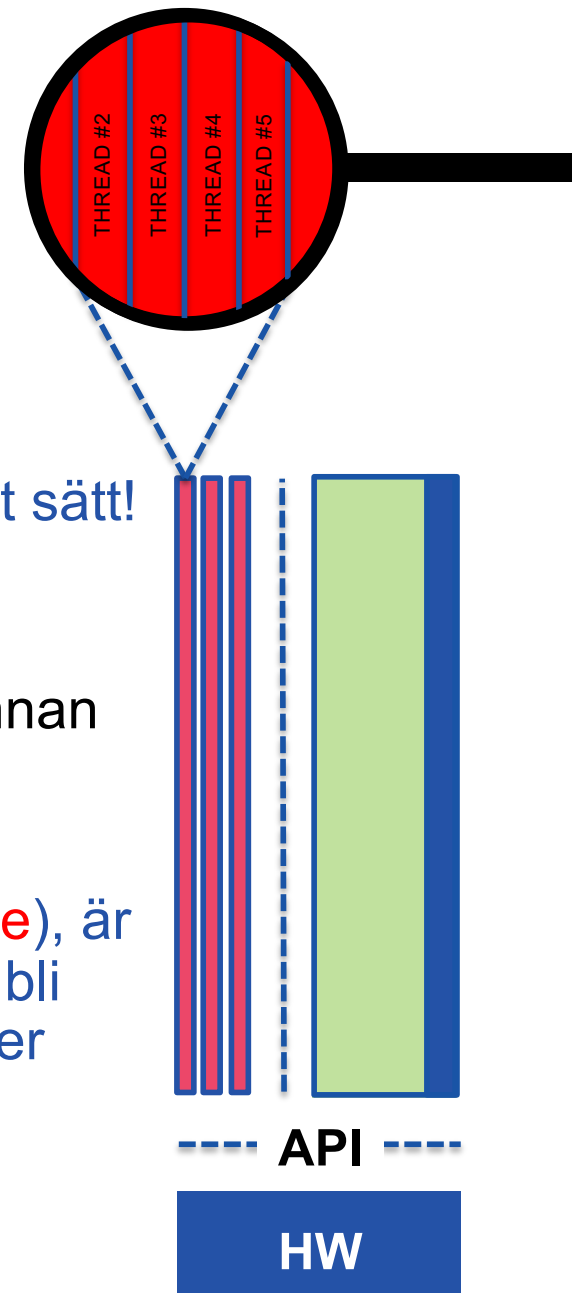
Bra, vi har Locks...

...som kan skydda datastrukturer...

...så de uppdateras på ett korrekt sätt!

Men i ett multitrådat program uppstår ofta behovet att en tråd måste vänta på att en annan tråd blir klar innan den kan/får fortsätta...

...att använda en variabel som "flagga" (**done**), är sällan en bra lösning då lösningen bara kan bli **while (!done)**; det skulle behövas mekanismer som tillät en tråd att vänta på att ett tillstånd inträffade på ett effektivt sätt!



Det behövs något som benämns: **Condition Variables!**



Condition Variables

Vi behöver en dylik variabel... `pthread_cond_t c;`

En **wait** funktion där tråden (effektivt) väntar på klartecken...

```
pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);
```

...som också använder ett lås, som förväntas vara taget,
och som wait släpper när tråden börjar vänta.

En **signal** funktion som kan signalerar klartecken...

```
pthread_cond_signal(pthread_cond_t *c);
```

...som "väcker" (en) eventuellt väntade tråd som återtar låset.

Vi testar det här med ett enkelt exempel:

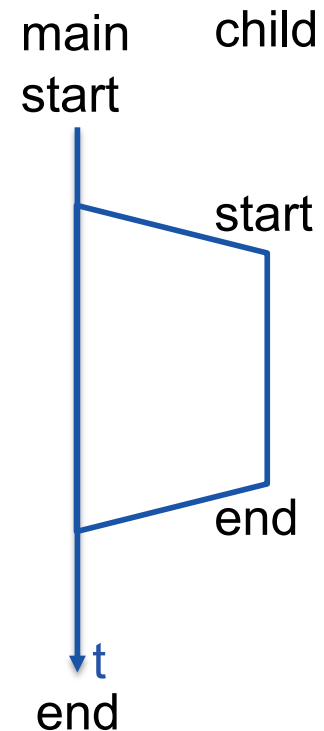


Condition Variables

```
volatile int done = 0;

void *child(void *arg) {
    printf("child\n");
    done = 1;
    return NULL;
}

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t c;
    Pthread_create(&c, NULL, child, NULL); // create child
    while (done == 0)
        ; // spin
    printf("parent: end\n");
    return 0;
}
```



parent: begin

child

parent: end

Vi vill att föräldern ska vänta på att "barntården" blir klar...

```

int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

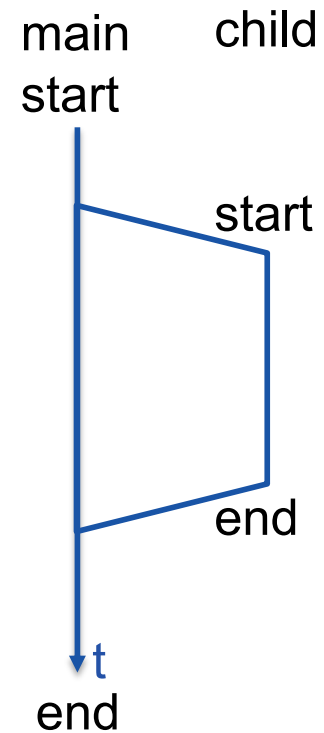
void thr_exit() {
    Pthread_mutex_lock(&m);
    done = 1;
    Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

void thr_join() {
    Pthread_mutex_lock(&m);
    while (done == 0)
        Pthread_cond_wait(&c, &m); (main blocked)
    Pthread_mutex_unlock(&m);
}

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    Pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}

```



parent: begin

```

int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
    Pthread_mutex_lock(&m);
    done = 1;
    Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}

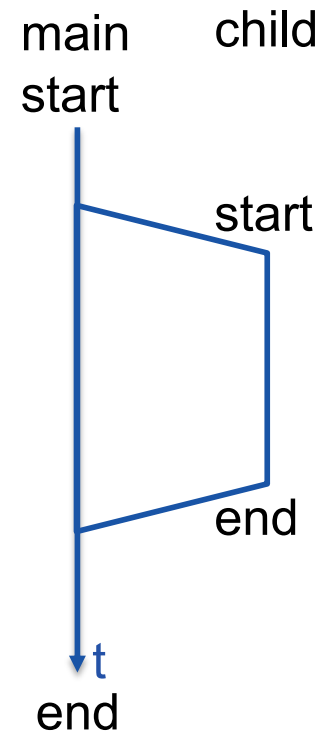
void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

void thr_join() {
    Pthread_mutex_lock(&m);
    while (done == 0)
        Pthread_cond_wait(&c, &m);
    Pthread_mutex_unlock(&m);
}

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    Pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}

```

(main ready)



parent: begin

child

```

int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

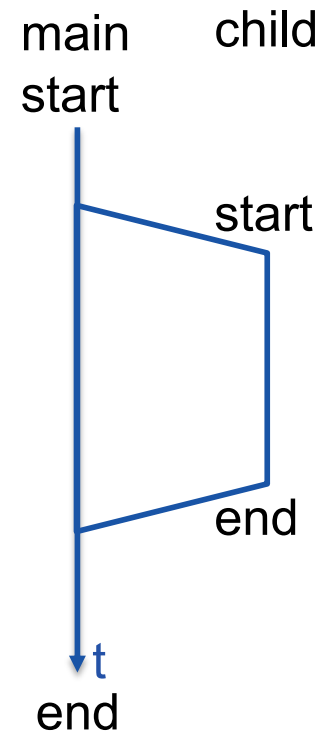
void thr_exit() {
    Pthread_mutex_lock(&m);
    done = 1;
    Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

void thr_join() {
    Pthread_mutex_lock(&m);
    while (done == 0)
        Pthread_cond_wait(&c, &m);
    Pthread_mutex_unlock(&m);
}

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    Pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}

```



parent: begin

child

parent: end

```

int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
    Pthread_mutex_lock(&m);
    done = 1;
    Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}

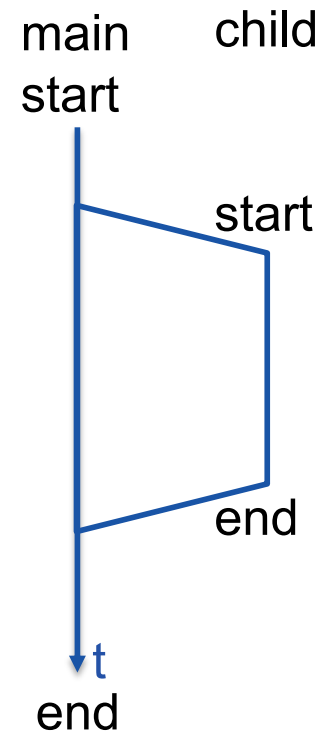
void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

void thr_join() {
    Pthread_mutex_lock(&m);
    while (done == 0)
        Pthread_cond_wait(&c, &m);
    Pthread_mutex_unlock(&m);
}

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    Pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}

```

"done är 1, no wait!"



(signal before wait, need done)

```

int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

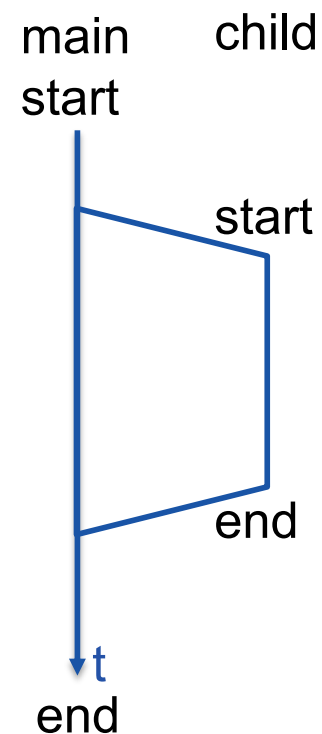
void thr_exit() {
    Pthread_mutex_lock(&m);
    done = 1;
    Pthread_cond_signal(&c);
    Pthread_mutex_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}

void thr_join() {
    Pthread_mutex_lock(&m);
    while (done == 0)
        Pthread_cond_wait(&c, &m);
    Pthread_mutex_unlock(&m);
}

int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    Pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}

```



(signal before wait, need lock)



Producent/Konsument (aka begränsad buffert)

```
int buffer;  
int count = 0; // initially, empty  
  
void put(int value) {  
    assert(count == 0);  
    count = 1;  
    buffer = value;  
}  
  
int get() {  
    assert(count == 1);  
    count = 0;  
    return buffer;  
}
```

Buffer

```
void *producer(void *arg) {  
    int i;  
    int loops = (int) arg;  
    for (i = 0; i < loops; i++) {  
        put(i);  
    }  
}  
  
void *consumer(void *arg) {  
    int i;  
    while (1) {  
        int tmp = get();  
        printf("%d\n", tmp);  
    }  
}
```

"Tillverka" en sak. "konsumera" saken...
"Tillverka" en sak. "konsumera" saken...
"Tillverka" en sak. "konsumera" saken...

Väldigt vanligt
Web/DB/Pipe



Producent/Konsument

```
cond_t cond;  
mutex_t mutex;
```

count

0

buffer

?

```
void *producer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        Pthread_mutex_lock(&mutex);           // p1  
        if (count == 1)                       // p2  
            Pthread_cond_wait(&cond, &mutex); // p3  
        put(i);                               // p4  
        Pthread_cond_signal(&cond);           // p5  
        Pthread_mutex_unlock(&mutex);         // p6  
    }  
}  
  
void *consumer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        Pthread_mutex_lock(&mutex);           // c1  
        if (count == 0)                       // c2  
            Pthread_cond_wait(&cond, &mutex); // c3  
        int tmp = get();                      // c4  
        Pthread_cond_signal(&cond);           // c5  
        Pthread_mutex_unlock(&mutex);         // c6  
        printf("%d\n", tmp);  
    }  
}
```



Producent/Konsument

```
cond_t cond;  
mutex_t mutex;
```

count 0 -> 1

buffer ? -> 0

```
void *producer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        Pthread_mutex_lock(&mutex);           // p1  
        if (count == 1)                       // p2  
            Pthread_cond_wait(&cond, &mutex); // p3  
        put(i);                               // p4  
        Pthread_cond_signal(&cond);           // p5  
        Pthread_mutex_unlock(&mutex);         // p6  
    }  
}
```

```
void *consumer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        Pthread_mutex_lock(&mutex);           // c1  
        if (count == 0)                       // c2  
            Pthread_cond_wait(&cond, &mutex); // c3  
        int tmp = get();                      // c4  
        Pthread_cond_signal(&cond);           // c5  
        Pthread_mutex_unlock(&mutex);         // c6  
        printf("%d\n", tmp);  
    }  
}
```



Producent/Konsument

```
cond_t cond;  
mutex_t mutex;
```

count

1 -> 0

buffer

Get 0

```
void *producer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        Pthread_mutex_lock(&mutex);           // p1  
        if (count == 1)                       // p2  
            Pthread_cond_wait(&cond, &mutex); // p3  
        put(i);                               // p4  
        Pthread_cond_signal(&cond);           // p5  
        Pthread_mutex_unlock(&mutex);         // p6  
    }  
}
```

```
void *consumer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        Pthread_mutex_lock(&mutex);           // c1  
        if (count == 0)                       // c2  
            Pthread_cond_wait(&cond, &mutex); // c3  
        int tmp = get();                      // c4  
        Pthread_cond_signal(&cond);           // c5  
        Pthread_mutex_unlock(&mutex);         // c6  
        printf("%d\n", tmp);  
    }  
}
```

```
cond_t cond;  
mutex_t mutex;
```

count 0 -> 1

buffer 0 -> 1

```
void *producer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        Pthread_mutex_lock(&mutex);           // p1  
        if (count == 1)                       // p2  
            Pthread_cond_wait(&cond, &mutex); // p3  
        put(i);                               // p4  
        Pthread_cond_signal(&cond);           // p5  
        Pthread_mutex_unlock(&mutex);         // p6  
    }  
}
```

```
void *consumer(void *arg) {  
    int i;  
    for (i = 0; i < loops; i++) {  
        Pthread_mutex_lock(&mutex);           // c1  
        if (count == 0)                       // c2  
            Pthread_cond_wait(&cond, &mutex); // c3  
        int tmp = get();                      // c4  
        Pthread_cond_signal(&cond);           // c5  
        Pthread_mutex_unlock(&mutex);         // c6  
        printf("%d\n", tmp);  
    }  
}
```



Producent/Konsument

Konsument #1 Konsument #2 Producent #1

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Running		Ready		Ready	0	
c2	Running		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep		Ready	p1	Running	0	
	Sleep		Ready	p2	Running	0	
	Sleep		Ready	p4	Running	1	Buffer now full
	Ready		Ready	p5	Running	1	T_{c1} awoken
	Ready		Ready	p6	Running	1	
	Ready		Ready	p1	Running	1	
	Ready		Ready	p2	Running	1	
	Ready		Ready	p3	Sleep	1	Buffer full; sleep
	Ready	c1	Running		Sleep	1	T_{c2} sneaks in ...
	Ready	c2	Running		Sleep	1	
	Ready	c4	Running		Sleep	0	... and grabs data
	Ready	c5	Running		Ready	0	T_p awoken
	Ready	c6	Running		Ready	0	
c4	Running		Ready		Ready	0	Oh oh! No data

```
if (count == 0)
    Pthread_cond_wait(&cond, &mutex);
```

Att en tråd väcks...

...är bara en hint!

```
while (count == 0)
    Pthread_cond_wait(&cond, &mutex);
```

(spurious wakeups)

T_{c1}	State	T_{c2}	State	T_p	State	Count	Comment
c1	Running		Ready		Ready	0	
c2	Running		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep	c1	Running		Ready	0	
	Sleep	c2	Running		Ready	0	
	Sleep	c3	Sleep		Ready	0	Nothing to get
	Sleep		Sleep	p1	Running	0	
	Sleep		Sleep	p2	Running	0	
	Sleep		Sleep	p4	Running	1	Buffer now full
	Ready		Sleep	p5	Running	1	T_{c1} awoken
	Ready		Sleep	p6	Running	1	
	Ready		Sleep	p1	Running	1	
	Ready		Sleep	p2	Running	1	
	Ready		Sleep	p3	Sleep	1	Must sleep (full)
c2	Running		Sleep		Sleep	1	Recheck condition
c4	Running		Sleep		Sleep	0	T_{c1} grabs data
c5	Running		Ready		Sleep	0	Oops! Woke T_{c2}
c6	Running		Ready		Sleep	0	
c1	Running		Ready		Sleep	0	
c2	Running		Ready		Sleep	0	
c3	Sleep		Ready		Sleep	0	Nothing to get
	Sleep	c2	Running		Sleep	0	
	Sleep	c3	Sleep		Sleep	0	Everyone asleep...

Problemet är att det bara finns en CV så det är 50% chans att Consumer #1 väcker Producenten (väcks Consumer #2 är det kört!)



```
cond_t  empty, fill;
mutex_t mutex;
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 1)
            Pthread_cond_wait(&empty, &mutex);
        put(i);
        Pthread_cond_signal(&fill);
        Pthread_mutex_unlock(&mutex);
    }
}
```



Kan bara väcka konsument, ok!

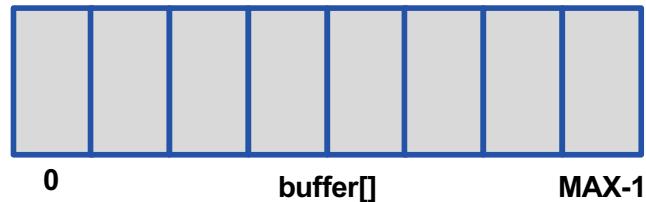
```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 0)
            Pthread_cond_wait(&fill, &mutex);
        int tmp = get();
        Pthread_cond_signal(&empty);
        Pthread_mutex_unlock(&mutex);
        printf("%d\n", tmp);
    }
}
```



Kan bara väcka producent, ok!



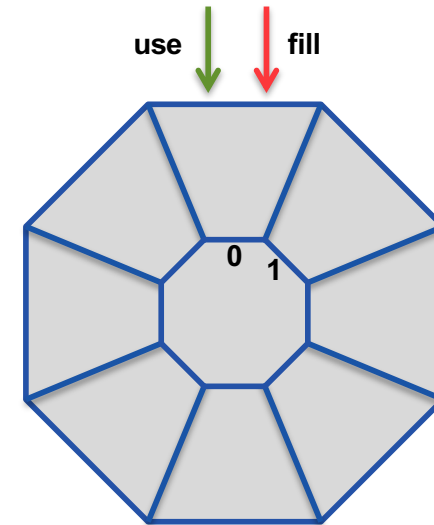
Producent/Konsument



```
int buffer[MAX];
int fill_ptr = 0;
int use_ptr = 0;
int count = 0;

void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}

int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```



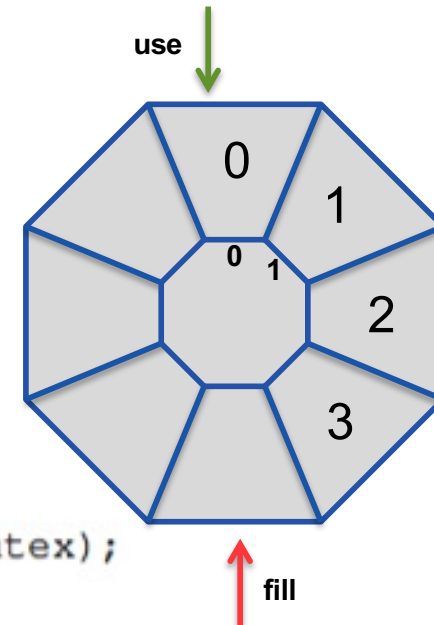
0
1 (2 3 4 5 6 7 0 1 2 3 4 5...)
1

Vi utökar vår buffert från 1 "plats" till "MAX" antal platser...

```
cond_t empty, fill;
mutex_t mutex;
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == MAX)
            Pthread_cond_wait(&empty, &mutex);
        put(i);
        Pthread_cond_signal(&fill);
        Pthread_mutex_unlock(&mutex);
    }
}
```

```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 0)
            Pthread_cond_wait(&fill, &mutex);
        int tmp = get();
        Pthread_cond_signal(&empty);
        Pthread_mutex_unlock(&mutex);
        printf("%d\n", tmp);
    }
}
```



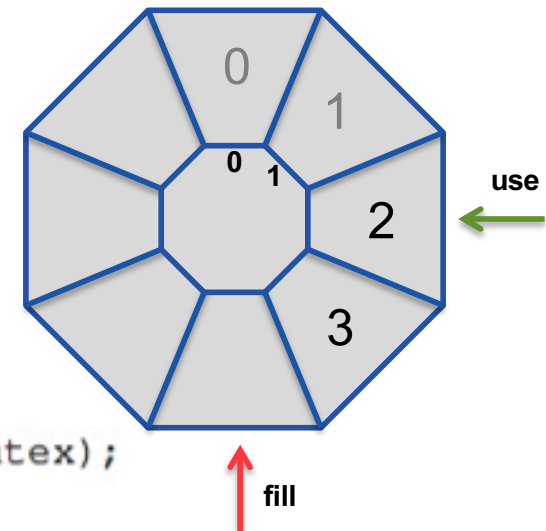
count == 4

Producent/Konsument

```
cond_t empty, fill;
mutex_t mutex;
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == MAX)
            Pthread_cond_wait(&empty, &mutex);
        put(i);
        Pthread_cond_signal(&fill);
        Pthread_mutex_unlock(&mutex);
    }
}
```

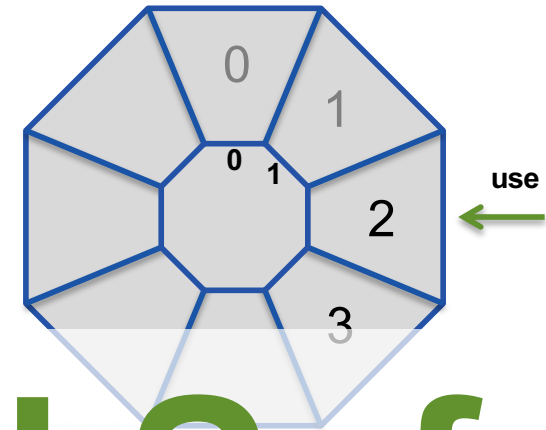
```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 0)
            Pthread_cond_wait(&fill, &mutex);
        int tmp = get();
        Pthread_cond_signal(&empty);
        Pthread_mutex_unlock(&mutex);
        printf("%d\n", tmp);
    }
}
```



count == 2

```
cond_t empty, fill;
mutex_t mutex;
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == MAX)
            Pthread_cond_wait(&empty, &mutex);
        out(i);
        Pthread_cond_signal(&fill);
        Pthread_mutex_unlock(&mutex);
    }
}
```



count == 2

Wow! Thread Safe Bounded Buffer

```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 0)
            Pthread_cond_wait(&fill, &mutex);
        int tmp = get();
        Pthread_cond_signal(&empty);
        Pthread_mutex_unlock(&mutex);
        printf("%d\n", tmp);
    }
}
```

Producer/Consumer



Ett sista problem...

```
// how many bytes of the heap are free?  
int bytesLeft = MAX_HEAP_SIZE;
```

```
// need lock and condition too  
cond_t c;  
mutex_t m;
```

```
void *  
allocate(int size) {  
    Pthread_mutex_lock(&m);  
    while (bytesLeft < size)  
        Pthread_cond_wait(&c, &m);  
    void *ptr = ...; // get mem from heap  
    bytesLeft -= size;  
    Pthread_mutex_unlock(&m);  
    return ptr;  
}
```

T1 T2



```
void free(void *ptr, int size) {  
    Pthread_mutex_lock(&m);  
    bytesLeft += size;  
    Pthread_cond_signal(&c); // whom to signal??  
    Pthread_mutex_unlock(&m);  
}
```

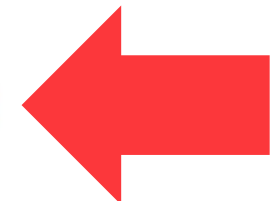
T3



T#1: alloc(100)
Not possible wait

T#2: alloc(50)
Not possible wait

T#3: free(75)
Signal (wake) ?



Nu är det inte längre en fråga om att väcka producent eller konsument...
...som vi kunde lösa med en till CV, utan att väcka "rätt" konsument?!



Ett sista problem...

```
// how many bytes of the heap are free?  
int bytesLeft = MAX_HEAP_SIZE;
```

```
// need lock and condition too  
cond_t c;  
mutex_t m;
```

```
void *  
allocate(int size) {  
    Pthread_mutex_lock(&m);  
    while (bytesLeft < size)  
        Pthread_cond_wait(&c, &m);  
    void *ptr = ...; // get mem from heap  
    bytesLeft -= size;  
    Pthread_mutex_unlock(&m);  
    return ptr;  
}
```

```
void free(void *ptr, int size) {  
    Pthread_mutex_lock(&m);  
    bytesLeft += size;  
Pthread_cond_signal(&c); // whom to signal??  
    Pthread_mutex_unlock(&m);  
}
```

T#1: alloc(100)
Not possible wait

T#2: alloc(50)
Not possible wait

T#3: free(75)
Signal (wake) ?

Väck alla!

`pthread_cond_broadcast()`

(Kommer att leda till många context switches!)

En tredje mekanism: Semaforer

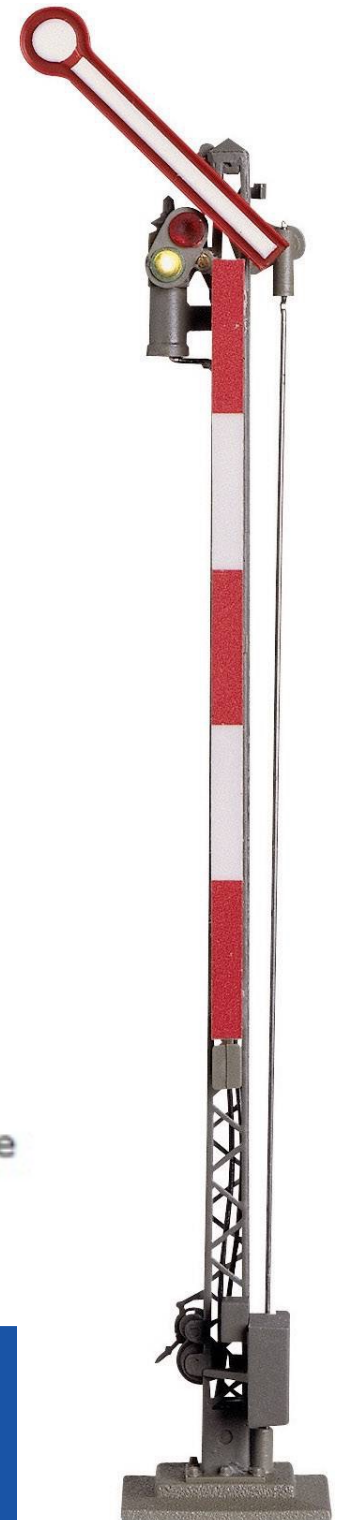
```
#include <semaphore.h>
sem_t s;
sem_init(&s, 0, 1);
```



```
int sem_wait(sem_t *s) {
    decrement the value of semaphore s by one
    wait if value of semaphore s is negative
}

int sem_post(sem_t *s) {
    increment the value of semaphore s by one
    if there are one or more threads waiting, wake one
}
```

$S > 0$ överskott/hur många som "använder" något
 $S < 0$ underskott/hur många som "väntar" på något





Semafor som Lock (aka binary semaphores)

```
sem_t m;  
sem_init(&m, 0, X);  
  
sem_wait(&m);  
// critical section here  
sem_post(&m);
```

Vad bör X rimligen vara?



Synkroniseringsproblemet, take 2

```
sem_t s;

void *
child(void *arg) {
    printf("child\n");
    sem_post(&s); // signal here: child is done
    return NULL;
}

int
main(int argc, char *argv[]) {
    sem_init(&s, 0, X); // what should X be?
    printf("parent: begin\n");
    pthread_t c;
    Pthread_create(&c, NULL, child, NULL);
    sem_wait(&s); // wait here for child
    printf("parent: end\n");
    return 0;
}
```

Vad bör X rimligen vara?



Synkroniseringsproblemet, take 2 BS

```
sem_t s;

void *
child(void *arg) {
    printf("child\n");
    sem_post(&s); // signal here: child is done
    return NULL;
}

int
main(int argc, char *argv[]) {
    sem_init(&s, 0, X); // what should X be?
    printf("parent: begin\n");
    pthread_t c;
    Pthread_create(&c, NULL, child, NULL);
    sem_wait(&s); // wait here for child
    printf("parent: end\n");
    return 0;
}
```

X=0, main kör till wait... ...ok!



Synkroniseringsproblemet, take 2 BS

```
sem_t s;

void *
child(void *arg) {
    printf("child\n");
    sem_post(&s); // signal here: child is done
    return NULL;
}

int
main(int argc, char *argv[]) {
    sem_init(&s, 0, X); // what should X be?
    printf("parent: begin\n");
    pthread_t c;
    Pthread_create(&c, NULL, child, NULL);
    sem_wait(&s); // wait here for child
    printf("parent: end\n");
    return 0;
}
```

X=0, main kör till create... ...ok!

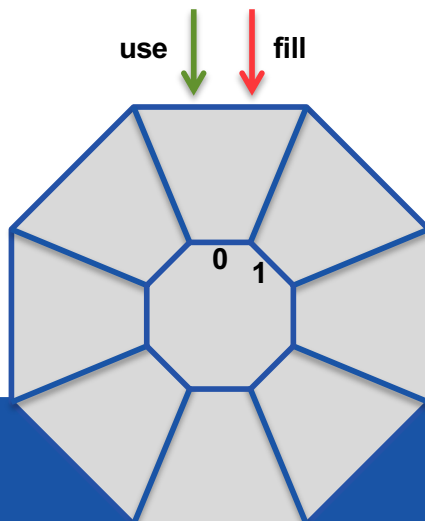
Producent/Konsument II (aka begränsad buffert)

```
int buffer[MAX];
int fill = 0;
int use = 0;

void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}

int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    return tmp;
}
```

empty == MAX
full == 0



```
sem_t empty;
sem_t full;

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}

void *consumer(void *arg) {
    int i, tmp = 0;
    while (tmp != -1) {
        sem_wait(&full);
        tmp = get();
        sem_post(&empty);
        printf("%d\n", tmp);
    }
}

int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // M
    sem_init(&full, 0, 0);    // .
    // ...
}
```

Bra, men...

Producent/Konsument II (aka begränsad buffert)

```
int buffer[MAX];
int fill = 0;
int use = 0;
```

```
void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}
```

```
int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    return tmp;
}
```

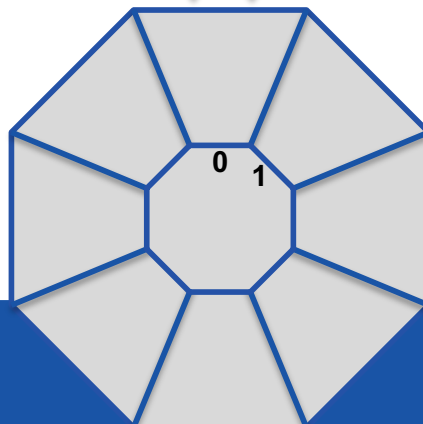
```
sem_t empty;
sem_t full;
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        put(i);
        sem_post(&full);
    }
}
```

```
void *consumer(void *arg) {
    int i, tmp = 0;
    while (tmp != -1) {
        sem_wait(&full);
        tmp = get();
        sem_post(&empty);
        printf("%d\n", tmp);
    }
}
```

```
int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // M
    sem_init(&full, 0, 0);    // .
    // ...
}
```

use ↓ fill ↓



empty == MAX
full == 0

Bra, men...



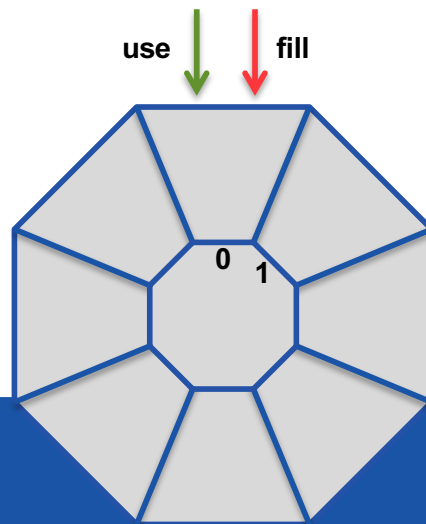
C/S aka BB, take II

```
int buffer[MAX];
int fill = 0;
int use = 0;

void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}

int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    return tmp;
}
```

empty == MAX
full == 0



```
sem_t empty;
sem_t full;
sem_t mutex;
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        sem_post(&full);
        sem_post(&mutex);
    }
}
```

```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int tmp = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("%d\n", tmp);
    }
}
```

```
int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // M
    sem_init(&full, 0, 0); // .
    sem_init(&mutex, 0, 1); // m
    // ...
}
```



C/S aka BB, take II

```
int buffer[MAX];
int fill = 0;
int use = 0;

void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}

int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    return tmp;
}
```

use ↓ fill ↓

```
sem_t empty;
sem_t full;
sem_t mutex;
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        sem_post(&full);
        sem_post(&mutex);
    }
}
```

```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int tmp = get();
        sem_post(&empty);
        sem_post(&mutex);
    }
}
```

empty =
full == (

DEADLOCK!

```
sem_init(&mutex, 0, 1);
// ...
```



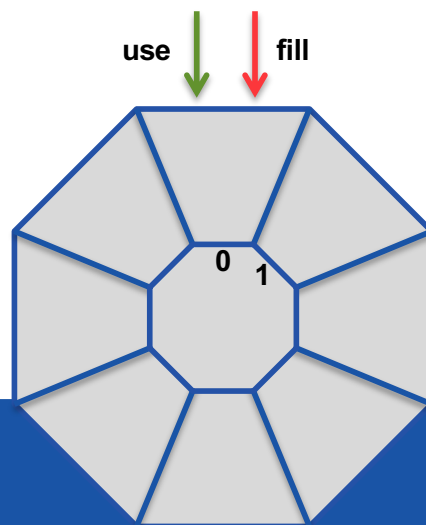
C/S aka BB, take II

```
int buffer[MAX];
int fill = 0;
int use = 0;

void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}

int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    return tmp;
}
```

empty == MAX
full == 0



```
sem_t empty;
sem_t full;
sem_t mutex;
```

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        put(i);
        sem_post(&mutex);
        sem_post(&full);
    }
}
```

```
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int tmp = get();
        sem_post(&mutex);
        sem_post(&empty);
        printf("%d\n", tmp);
    }
}
```

Ta inte låset för tidigt!

```
int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX); // M
    sem_init(&full, 0, 0);    // .
    sem_init(&mutex, 0, 1);  // m
    // ...
}
```



Reader-Writers Lock

```
typedef struct _rwlock_t {
    sem_t lock;          // binary semaphore (basic lock)
    sem_t writelock;     // used to allow ONE writer or MANY readers
    int   readers;       // count of readers reading in critical section
} rwlock_t;
```

```
void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}
```

0 läsare
(Går att skriva)
0 skrivare

```
void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1)
        sem_wait(&rw->writelock); // first reader acquires writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0)
        sem_post(&rw->writelock); // last reader releases writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}
```

```
void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}
```



Reader-Writers Lock

```
typedef struct _rwlock_t {
    sem_t lock;          // binary semaphore (basic lock)
    sem_t writelock;     // used to allow ONE writer or MANY readers
    int   readers;       // count of readers reading in critical section
} rwlock_t;
```

```
void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}
```

En läsare
(Går inte att skriva)
0 skrivare

```
void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1)
        sem_wait(&rw->writelock); // first reader acquires writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0)
        sem_post(&rw->writelock); // last reader releases writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}
```

```
void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}
```



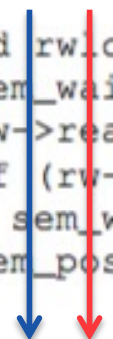
Reader-Writers Lock

```
typedef struct _rwlock_t {
    sem_t lock;          // binary semaphore (basic lock)
    sem_t writelock;     // used to allow ONE writer or MANY readers
    int   readers;       // count of readers reading in critical section
} rwlock_t;
```

```
void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}
```

Två läsare
(Går inte att skriva)
0 skrivare

```
void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1)
        sem_wait(&rw->writelock); // first reader acquires writelock
    sem_post(&rw->lock);
}
```



```
void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0)
        sem_post(&rw->writelock); // last reader releases writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}
```

```
void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}
```



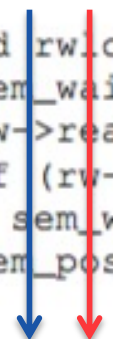
Reader-Writers Lock

```
typedef struct _rwlock_t {
    sem_t lock;          // binary semaphore (basic lock)
    sem_t writelock;     // used to allow ONE writer or MANY readers
    int   readers;       // count of readers reading in critical section
} rwlock_t;
```

```
void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}
```

Två läsare
(Går inte att skriva)
En skrivare blockerad

```
void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1)
        sem_wait(&rw->writelock); // first reader acquires writelock
    sem_post(&rw->lock);
}
```



```
void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0)
        sem_post(&rw->writelock); // last reader releases writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}
```



```
void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}
```



Reader-Writers Lock

```
typedef struct _rwlock_t {
    sem_t lock;          // binary semaphore (basic lock)
    sem_t writelock;     // used to allow ONE writer or MANY readers
    int   readers;       // count of readers reading in critical section
} rwlock_t;
```

```
void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}
```

En läsare
(Går inte att skriva)
En skrivare blockerad

```
void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1)
        sem_wait(&rw->writelock); // first reader acquires writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0)
        sem_post(&rw->writelock); // last reader releases writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}
```

```
void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}
```



Reader-Writers Lock

```
typedef struct _rwlock_t {
    sem_t lock;          // binary semaphore (basic lock)
    sem_t writelock;     // used to allow ONE writer or MANY readers
    int   readers;       // count of readers reading in critical section
} rwlock_t;
```

```
void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}
```

0 läsare
(Går att skriva)
En skrivare "ready"

```
void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1)
        sem_wait(&rw->writelock); // first reader acquires writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0)
        sem_post(&rw->writelock); // last reader releases writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}
```

```
void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}
```



Reader-Writers Lock

```
typedef struct _rwlock_t {
    sem_t lock;          // binary semaphore (basic lock)
    sem_t writelock;     // used to allow ONE writer or MANY readers
    int   readers;       // count of readers reading in critical section
} rwlock_t;
```

```
void rwlock_init(rwlock_t *rw) {
    rw->readers = 0;
    sem_init(&rw->lock, 0, 1);
    sem_init(&rw->writelock, 0, 1);
}
```

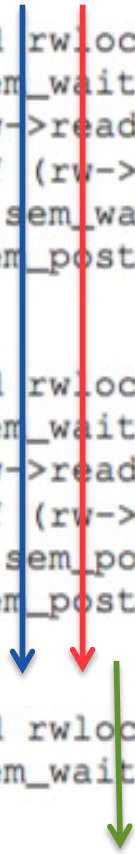
0 läsare
(Går att skriva)
En skrivare "running"

```
void rwlock_acquire_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers++;
    if (rw->readers == 1)
        sem_wait(&rw->writelock); // first reader acquires writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_release_readlock(rwlock_t *rw) {
    sem_wait(&rw->lock);
    rw->readers--;
    if (rw->readers == 0)
        sem_post(&rw->writelock); // last reader releases writelock
    sem_post(&rw->lock);
}
```

```
void rwlock_acquire_writelock(rwlock_t *rw) {
    sem_wait(&rw->writelock);
}
```

```
void rwlock_release_writelock(rwlock_t *rw) {
    sem_post(&rw->writelock);
}
```





Reader-Writers Lock

```
typedef struct _rwlock_t {  
    sem_t lock;        // binary semaphore (basic lock)  
    sem_t writelock;    // used to allow ONE writer or MANY readers  
    int   readers;      // count of readers reading in critical section  
} rwlock_t;
```

```
void rwlock_init(rwlock_t *rw) {  
    rw->readers = 0;  
    sem_init(&rw->lock, 0, 1);  
    sem_init(&rw->writelock, 0, 1);  
}
```

0 läsare
(Går att skriva)
0 skrivare

```
void rwlock_acquire_readlock(rwlock_t *rw) {  
    sem_wait(&rw->lock);  
    rw->readers++;  
    if (rw->readers == 1)  
        sem_wait(&rw->writelock); // first reader acquires writelock  
    sem_post(&rw->lock);  
}
```

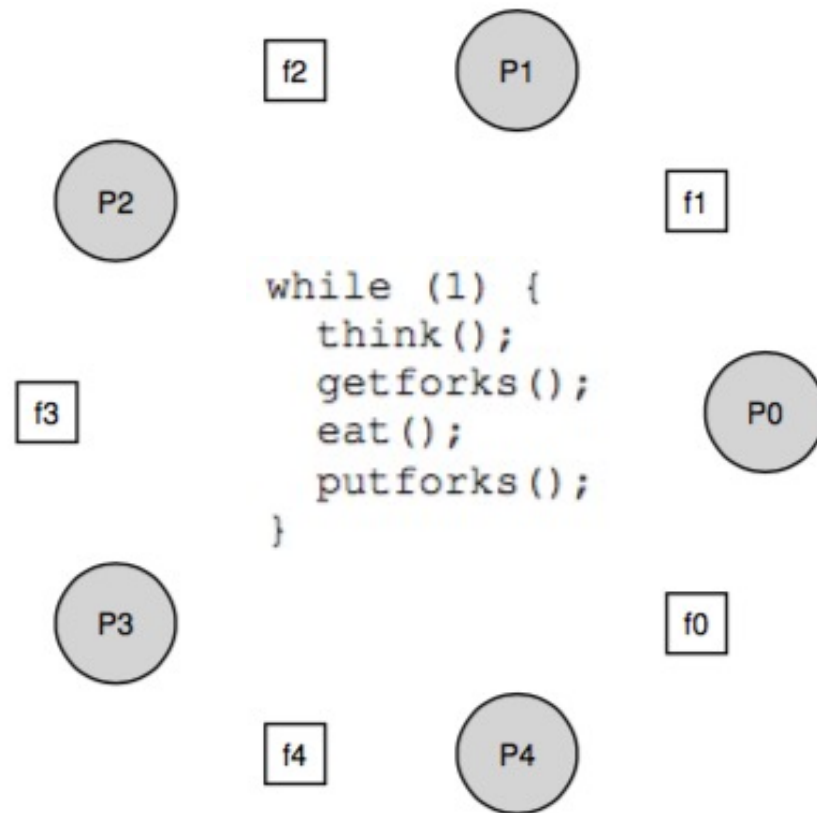
```
void rwlock_release_readlock(rwlock_t *rw) {  
    sem_wait(&rw->lock);  
    rw->readers--;  
    if (rw->readers == 0)  
        sem_post(&rw->writelock); // last reader releases writelock  
    sem_post(&rw->lock);  
}
```

```
void rwlock_acquire_writelock(rwlock_t *rw) {  
    sem_wait(&rw->writelock);  
}
```

```
void rwlock_release_writelock(rwlock_t *rw) {  
    sem_post(&rw->writelock);  
}
```

Rättvis?

The Dining Philosophers



```
while (1) {  
    think();  
    getforks();  
    eat();  
    putforks();  
}
```

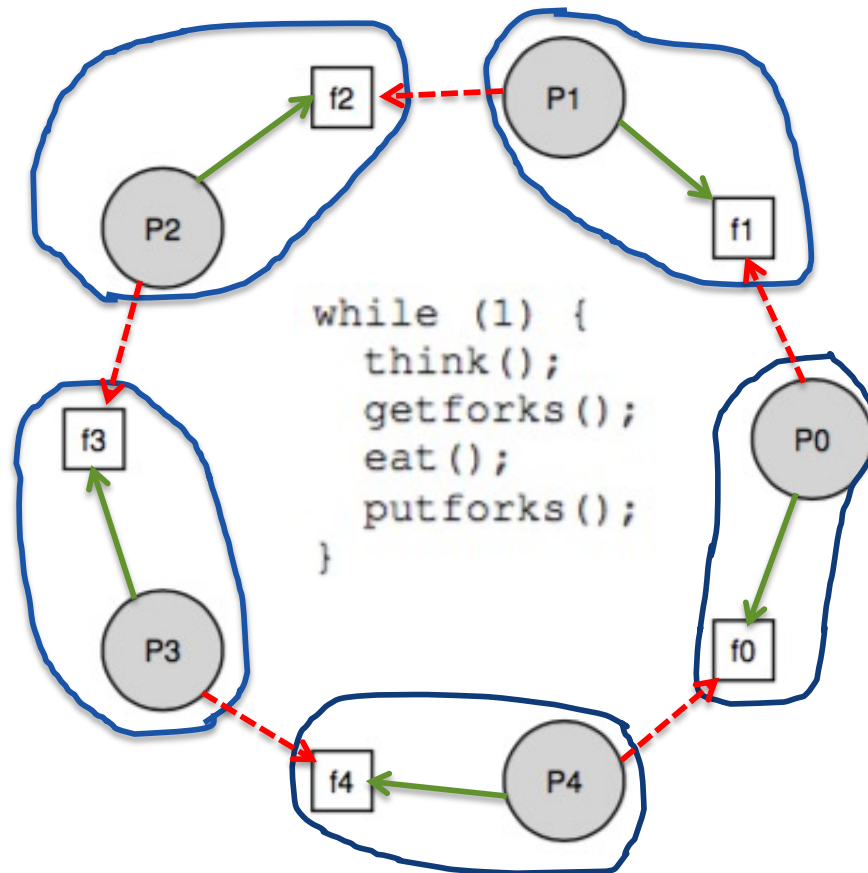
```
void getforks() {  
    sem_wait(forks[left(p)]);  
    sem_wait(forks[right(p)]);  
}
```

```
void putforks() {  
    sem_post(forks[left(p)]);  
    sem_post(forks[right(p)]);  
}
```

Vad händer om vi kör
detta på en cpu med
5 kärnor?!

```
int left(int p) { return p; }  
int right(int p) { return (p + 1) % 5; }
```

The Dining Philosophers



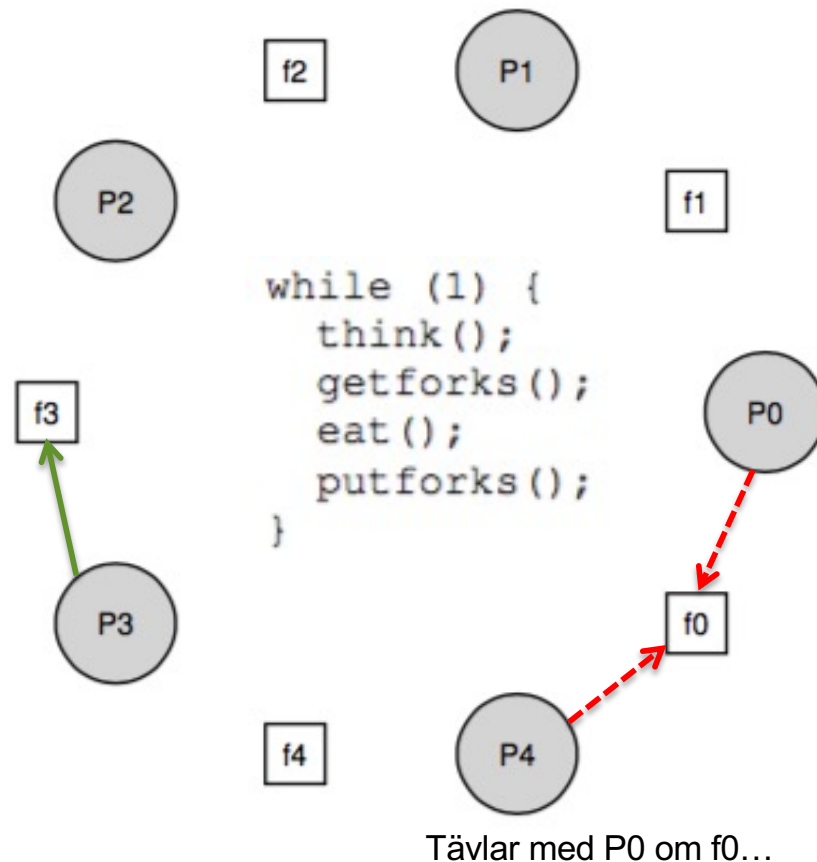
```
void getforks() {  
    sem_wait(forks[left(p)]);  
    sem_wait(forks[right(p)]);  
}
```

```
void putforks() {  
    sem_post(forks[left(p)]);  
    sem_post(forks[right(p)]);  
}
```

DEADLOCK!

```
int left(int p) { return p; }  
int right(int p) { return (p + 1) % 5; }
```

The Dining Philosophers



Tar f3 först...

Tävlrar med P0 om f0...

```
void getforks() {
    sem_wait(forks[left(p)]);
    sem_wait(forks[right(p)]);
}
```

```
void putforks() {
    sem_post(forks[left(p)]);
    sem_post(forks[right(p)]);
}
```

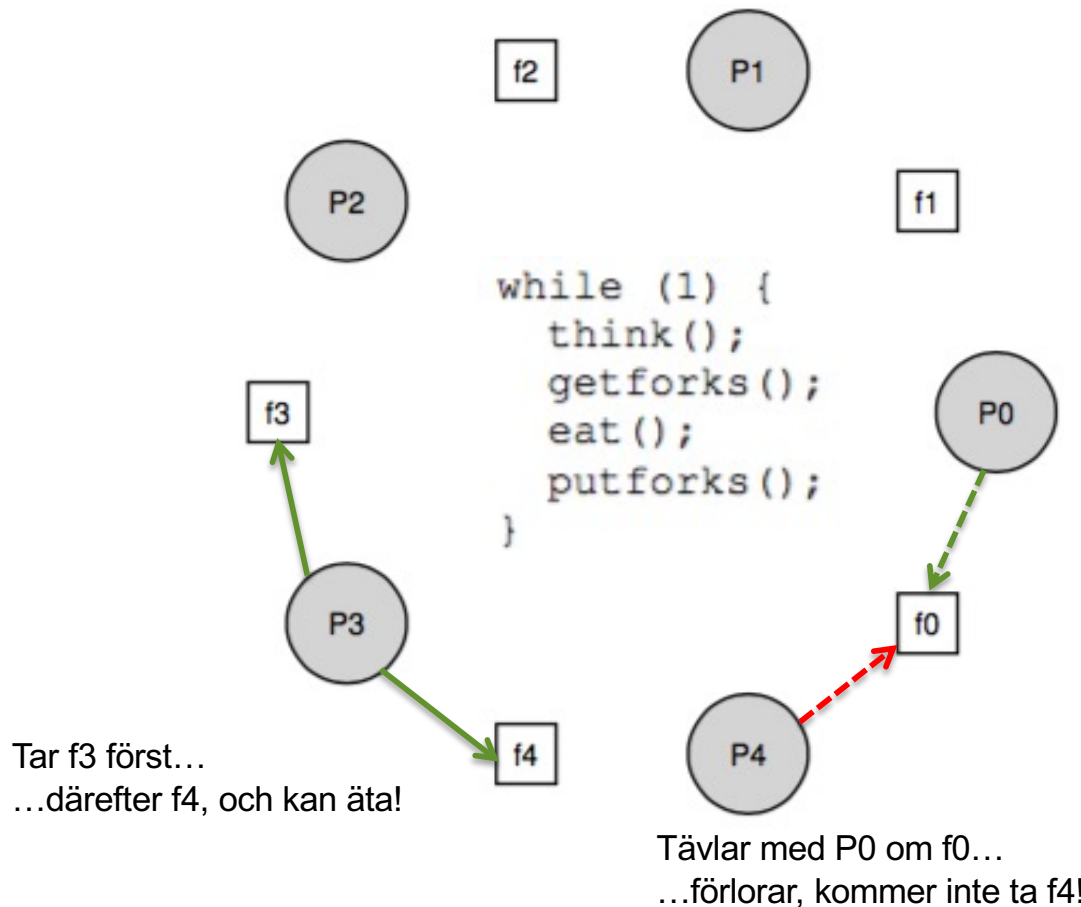


```
void getforks() {
    if (p == 4) {
        sem_wait(forks[right(p)]);
        sem_wait(forks[left(p)]);
    } else {
        sem_wait(forks[left(p)]);
        sem_wait(forks[right(p)]);
    }
}
```

```
int left(int p) { return p; }
int right(int p) { return (p + 1) % 5; }
```

Vi återkommer till
DEADLOCK!

The Dining Philosophers



```
void getforks() {
    sem_wait(forks[left(p)]);
    sem_wait(forks[right(p)]);
}
```

```
void putforks() {
    sem_post(forks[left(p)]);
    sem_post(forks[right(p)]);
}
```

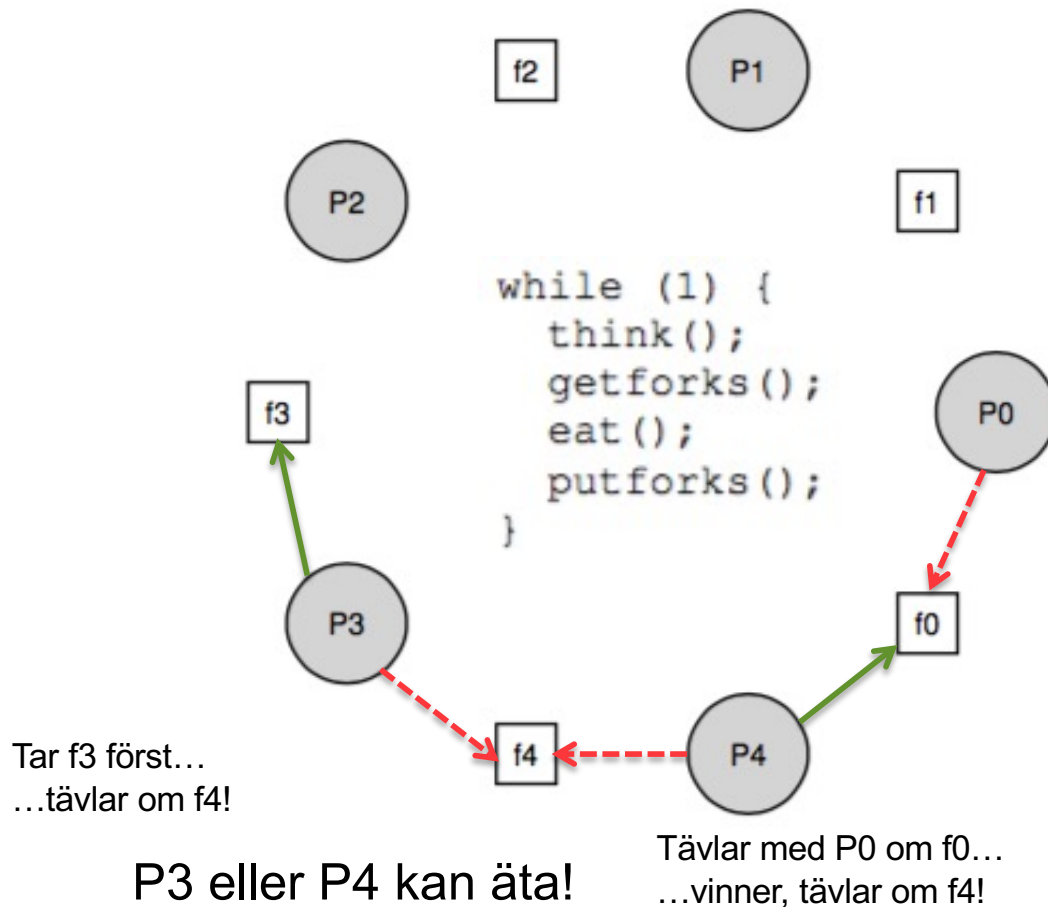


```
void getforks() {
    if (p == 4) {
        sem_wait(forks[right(p)]);
        sem_wait(forks[left(p)]);
    } else {
        sem_wait(forks[left(p)]);
        sem_wait(forks[right(p)]);
    }
}
```

```
int left(int p) { return p; }
int right(int p) { return (p + 1) % 5; }
```

Vi återkommer till
DEADLOCK!

The Dining Philosophers



```
void getforks() {
    sem_wait(forks[left(p)]);
    sem_wait(forks[right(p)]);
}
```

```
void putforks() {
    sem_post(forks[left(p)]);
    sem_post(forks[right(p)]);
}
```



```
void getforks() {
    if (p == 4) {
        sem_wait(forks[right(p)]);
        sem_wait(forks[left(p)]);
    } else {
        sem_wait(forks[left(p)]);
        sem_wait(forks[right(p)]);
    }
}
```

```
int left(int p) { return p; }
int right(int p) { return (p + 1) % 5; }
```

Vi återkommer till
DEADLOCK!



Parallellism är uppenbart knepigt...

Application	What it does	Non-Deadlock	Deadlock
MySQL	Database Server	14	9
Apache	Web Server	13	4
Mozilla	Web Browser	41	16
OpenOffice	Office Suite	6	2
Total		74	31

...97% av, non-deadlock felen, tillhörde 2 klasser av fel:



1: Atomicity-Violation Bugs...

```
Thread 1::  
if (thd->proc_info) {  
    ...  
    fputs(thd->proc_info, ...);  
    ...  
}
```

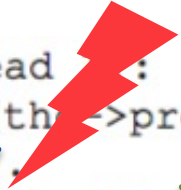
```
Thread 2::  
thd->proc_info = NULL;
```

Vad är det (eventuella) problemet?



1: Atomicity-Violation Bugs...

```
Thread 1:  
if (thd->proc_info) {  
    ...  
    fputs(thd->proc_info, ...);  
    ...  
}  
  
Thread 2::  
thd->proc_info = NULL;
```

  **Segmentation Fault...**



Vad är det (eventuella) problemet?

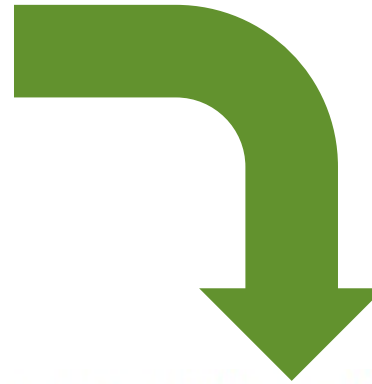
Koden är skriven som om: `if () {}` är en atomisk operation, men det är den inte och det kan följaktligen bli helt fel!



1: Atomicity-Violation Bugs...

```
Thread 1::  
if (thd->proc_info) {  
    ...  
    fputs(thd->proc_info, ...);  
    ...  
}
```

```
Thread 2::  
thd->proc_info = NULL;
```



```
pthread_mutex_t proc_info_lock = PTHREAD_MUTEX_INITIALIZER;
```

```
Thread 1::  
pthread_mutex_lock(&proc_info_lock);  
if (thd->proc_info) {  
    ...  
    fputs(thd->proc_info, ...);  
    ...  
}  
pthread_mutex_unlock(&proc_info_lock);
```

```
Thread 2::  
pthread_mutex_lock(&proc_info_lock);  
thd->proc_info = NULL;  
pthread_mutex_unlock(&proc_info_lock);
```



2: Order-Violation Bugs...

```
Thread 1::  
void init() {  
    ...  
    mThread = PR_CreateThread(mMain, ...);  
    ...  
}  
  
Thread 2::  
void mMain(...) {  
    ...  
    mState = mThread->State;  
    ...  
}
```

Vad är det (eventuella) problemet?



2: Order-Violation Bugs...

```
Thread 1:  
void init() {  
    ...  
    mThread = PR_CreateThread(mMain, ...);  
    ...  
}  
  
Thread 2::  
void mMain(...) {  
    ...  
    mState = mThread->State;  
    ...  
}
```

➡ **Segmentation Fault**
(or worse)!

Vad är det (eventuella) problemet?

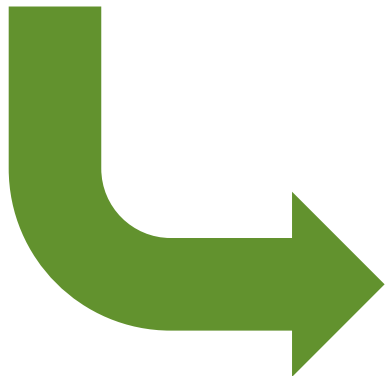
Koden är skriven som om det fanns en förväntad sekvens, som instruktionerna skulle utföras i, vilket inte är säkert!



2: Order-Violation Bugs...

```
Thread 1::
void init() {
    ...
    mThread = PR_CreateThread(mMain, ...);
    ...
}

Thread 2::
void mMain(...) {
    ...
    mState = mThread->State;
    ...
}
```



```
pthread_mutex_t mtLock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  mtCond = PTHREAD_COND_INITIALIZER;
int mtInit          = 0;
```

```
Thread 1::
void init() {
    ...
    mThread = PR_CreateThread(mMain, ...);

    // signal that the thread has been created...
    pthread_mutex_lock(&mtLock);
    mtInit = 1;
    pthread_cond_signal(&mtCond);
    pthread_mutex_unlock(&mtLock);
    ...
}
```

```
Thread 2::
void mMain(...) {
    ...
    // wait for the thread to be initialized...
    pthread_mutex_lock(&mtLock);
    while (mtInit == 0)
        pthread_cond_wait(&mtCond, &mtLock);
    pthread_mutex_unlock(&mtLock);

    mState = mThread->State;
    ...
}
```



Deadlock related problem...

Thread 1:
`pthread_mutex_lock (L1);`
`pthread_mutex_lock (L2);`

Thread 2:
`pthread_mutex_lock (L2);`
`pthread_mutex_lock (L1);`

- **Mutual exclusion:** Threads claim exclusive control of resources that they require (e.g., a thread grabs a lock).
- **Hold-and-wait:** Threads hold resources allocated to them (e.g., locks that they have already acquired) while waiting for additional resources (e.g., locks that they wish to acquire).
- **No preemption:** Resources (e.g., locks) cannot be forcibly removed from threads that are holding them.
- **Circular wait:** There exists a circular chain of threads such that each thread holds one or more resources (e.g., locks) that are being requested by the next thread in the chain.

Prevention – Avoidance – Detect & Recover



Prevention: Circular wait (bryter krav #4)

Ansats: Skriv kod som aldrig kan orsaka circular wait!

Säkerställs genom att alltid efterfråga lås i samma ordning!

Kräver djup insikt i kodbasen, inget stöd att det efterlevs!

Ibland kan partiell ordning räcka där sekvenser av låskedjor alltid utför i en specificerad ordning, utan att varje sekvens har placerats in i en helhetsbild där alla tänkbara förlopp analyserats.

Trick: Använd låsets adress (inte namn) för att säkerställa en viss ordning (high-to-low or low-to-high).

```
if (m1 > m2) { // grab locks in high-to-low address order
    pthread_mutex_lock(m1);
    pthread_mutex_lock(m2);
} else {
    pthread_mutex_lock(m2);
    pthread_mutex_lock(m1);
}
```



Prevention: Hold&wait (bryter krav #2)

Ansats: Skriv kod som aldrig inte håller "alla" lås!

Säkerställs genom att alltid efterfråga "alla" lås atomärt!

Kräver djup insikt i kodbasen, inget stöd att det efterlevs!

Mindre bra av flera skäl. På grund av hur kod utvecklas så är det svårt att identifiera alla lås som krävs. Metoden minskar också möjligheten att kod verkligen kan utföras parallellt.

Trick: Använd ett extra lås när lås ska efterfrågas!

```
pthread_mutex_lock(prevention);    // begin lock acquisition
pthread_mutex_lock(L1);
pthread_mutex_lock(L2);
...
pthread_mutex_unlock(prevention); // end
```



Prevention: No preemption (bryter krav #3)

Ansats: Skriv kod som aldrig håller lås om blocked!

Säkerställs genom test-and-set metod för att ta lås!

Kräver djup insikt i kodbasen, inget stöd att det efterlevs!

Mindre bra av flera skäl. På grund av hur kod utvecklas så är det svårt att identifiera alla lås som krävs. Det kan också vara svårt att återställa/hoppa tillbaks om låset inte kunde tas.

Trick: Använd ny låsmetod: trylock() – tar, om det går, no wait!

(LiveLock – very unlikely, delay)

```
top:
    pthread_mutex_lock(L1);
    if (pthread_mutex_trylock(L2) != 0) {
        pthread_mutex_unlock(L1);
        goto top;
    }
```



Prevention: Mutual Excl. (bryter krav #1)

Ansats: Skriv kod som inte behöver lås!!!

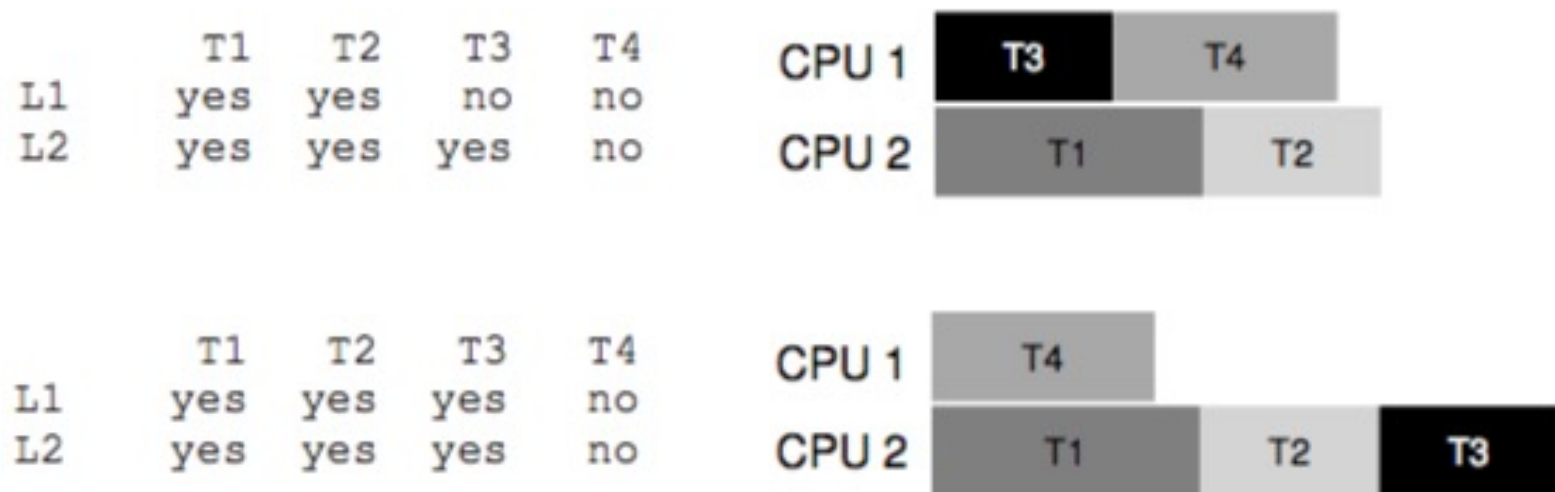
Säkerställs genom (hårdvarustödda) specialinstruktioner!

Krav att kompilatorn genererar kod med de "lås-fria" inst.

Säkerställer möjligheten till parallellism men kan kräva att vissa partier skrivs i assembler.

<code>x.fetch_and_store(y)</code>	do $x=y$ and return the old value of x
<code>x.fetch_and_add(y)</code>	do $x+=y$ and return the old value of x
<code>x.compare_and_swap(y, z)</code>	if x equals z , then do $x=y$. In either case, return old value of x .

Avoidance via Scheduling



Att förutse detta är i praktiken ytterst svårt, förutom i specialfall.
Används bl.a. i en del inbyggda system. (Dijkstra's Bankers alg.)



Detection and recovery

Om deadlock inträffar ytterst sällsynt...

...kanske en gång per år...

...låt det hända och tryck på reset!

Om möjligt skriv en bakgrundsprocess...

...som periodiskt analyserar resursbilden...

...finns det låsningar, starta automatiskt* om!

*kan kräva att information återställ, något som ibland kan kräva manuellt stöd (t.ex: återläsning från back-up filer)!



Begrepp

Condition Variables
Wait & Signal
Spurious wakeups

Bounded Buffer

Semaphores

Reader-Writers Lock

Deadlock
Mutual exclusion
Hold-and-wait
No preemption
Circular Wait

Prevention,
Avoidance,
Detection &
Recovery





Historik

1810xx AC 0.8 Skapad, saknas Bankers alg/övning.

1812xx AC 0.9 Begrepp. Flytta CV till F5?